

< Hashmap >

values can be duplicate

[ keys and values ]

- + only unique keys
- + insertion  $\rightarrow O(1)$
- + search  $\rightarrow O(1)$

Hotel room  $\rightarrow$

< Int, BOOLEAN >

400 true

401 false

402 false

403 true

population of all countries  $\rightarrow$  < string, long >

India 1.3B

China 1.5B

,

}

population of states of each country

map < String , < String , long > >  
          ↓                  ↓  
          Country          State  
          name             name

India

Bihar → x

WB → y

UP → z

⋮

USA

NY → a

Texas → b

Michigan → c

Data → <val, name>

range →  $1 \leq A[i] \leq 10^9$

no. of rows →  $1 \leq N \leq 10^5$

$10^9$  → create an array  $[10^9]$

roll  $\Rightarrow 1 \leq \text{roll} \leq 100$ .

no. of students  $\Rightarrow [0 \text{ to } 17]$

↓

to store  $\Rightarrow \text{arr}[17] \rightarrow \text{id} \Rightarrow [0-16]$

max 17

student  $\rightarrow$

roll no.  $\Rightarrow \underline{\underline{[0-16]}}$

$\left. \begin{array}{l} -50000 \\ -10 \\ 9 \\ 5 \\ 3 \\ 9 \\ 10 \\ 11 \end{array} \right\}$

$\frac{1}{10} \Rightarrow \underline{\underline{[0-9]}}$

arr[17]  $\Rightarrow$

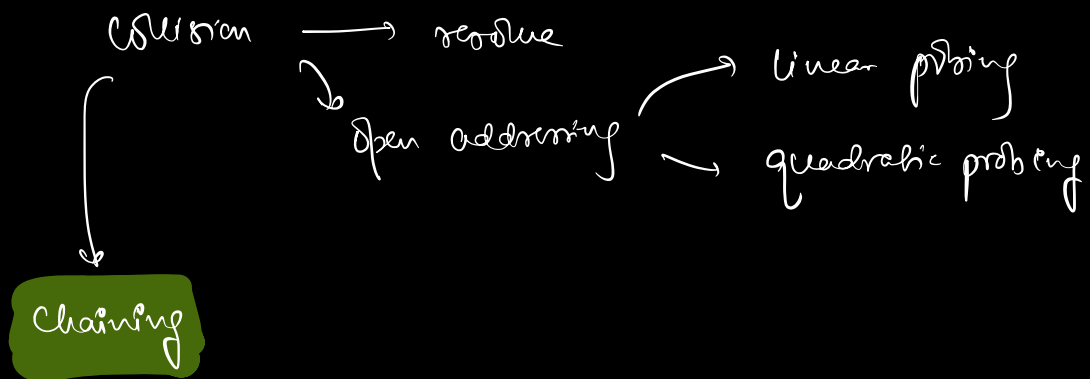
|    |    |    |                            |    |    |   |   |   |   |                               |
|----|----|----|----------------------------|----|----|---|---|---|---|-------------------------------|
| 0  | 1  | 2  | 3                          | 4  | 5  | 6 | 7 | 8 | 9 | 10                            |
|    |    |    | $\downarrow$               |    |    |   |   |   |   | $\downarrow$                  |
|    |    |    | $\langle 20, Anna \rangle$ |    |    |   |   |   |   | $\langle 10, Niladri \rangle$ |
|    |    |    |                            |    |    |   |   |   |   | $\uparrow$                    |
| 11 | 12 | 13 | 14                         | 15 | 16 |   |   |   |   |                               |

$20 \rightarrow 10 \% 17 \Rightarrow 10$   
 $20 \rightarrow 20 \% 17 \Rightarrow 3$   
 $27 \rightarrow 27 \% 17 \Rightarrow 10$

collision

$\langle 27, vkr \rangle$

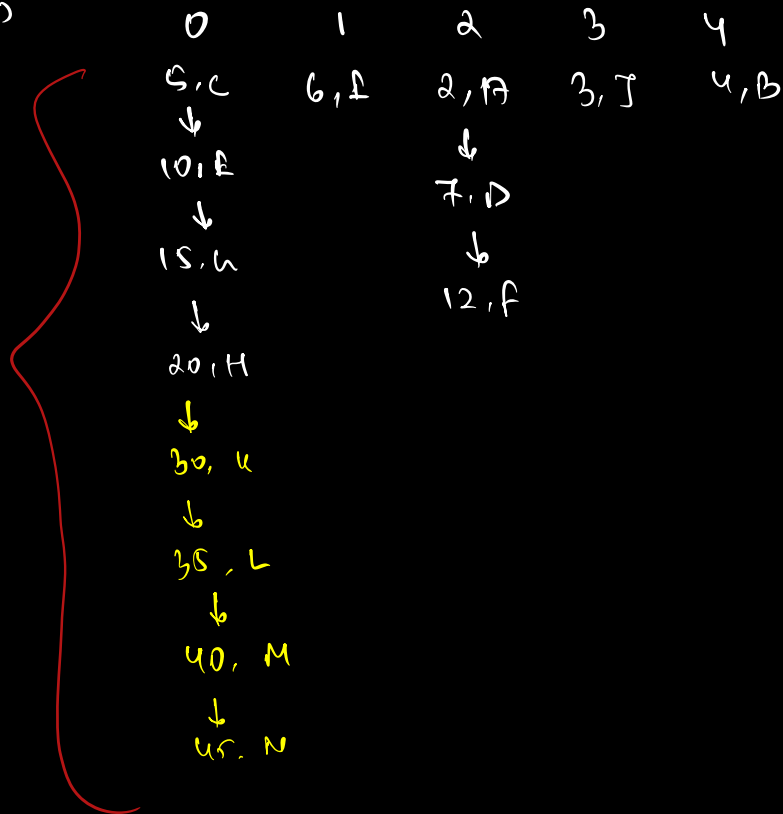
key  $\rightarrow$   $\boxed{\text{hash func.}} \rightarrow \text{key \% M} \rightarrow \text{O/p}$



arr  $\Rightarrow$  2, A    4, B    5, C    7, D    10, E, 12, F    15, G    20, H

6, I    3, J    M = 8    ,    30    35    40    45

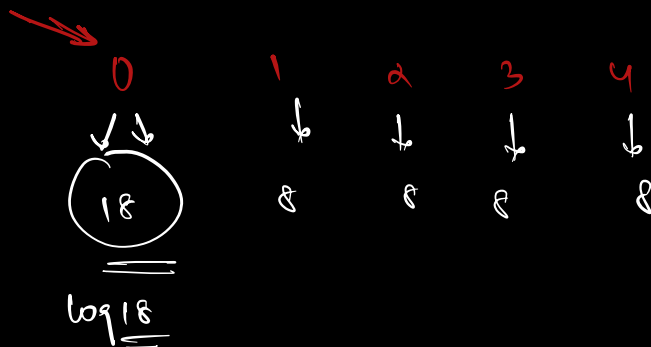
store  $\Rightarrow$



sol  $\Rightarrow$  12  $\rightarrow$  12, F  $\Rightarrow$  2  $\rightarrow$  F

when LL grows beyond a fixed length

LL  $\rightarrow$  B BST (Balanced Binary search tree)

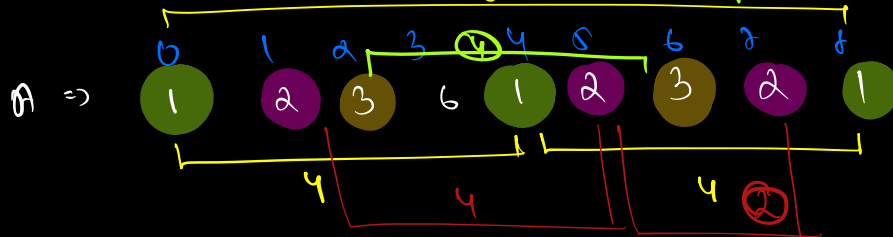


20% ( $\log 18$ ) + 80% ( $O(1)$ )

$\Rightarrow \underline{O(1)}$

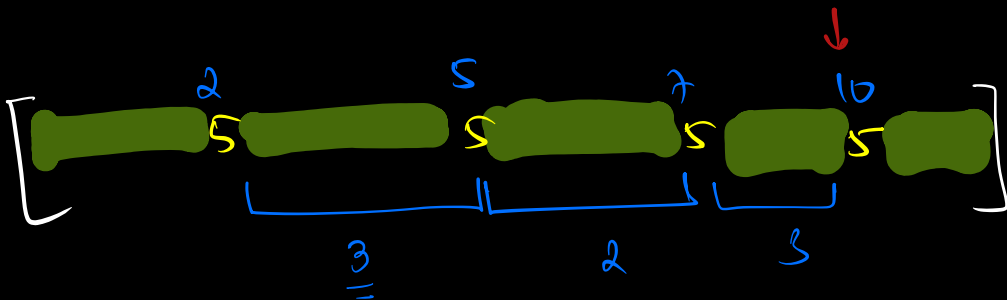
Closest duplicates

Q1. Given an array  $arr[N]$ , find any pair  $(i, j)$  such that  $j > i$ ,  $arr[i] = arr[j]$ , find min possible  $(j-i)$



$O(p^2) \underline{\underline{2}}$

Brute  $\rightarrow$  Iterate all pairs  $\rightarrow O(N^2)$



$O(p) \underline{\underline{2}}$

0 1 2 3 4 5 6 7 8  
1 2 3 6 1 2 3 2 1



|     |                             |
|-----|-----------------------------|
| ele | last                        |
| 1   | <del>5</del> 4 8            |
| 2   | <del>4</del> <del>5</del> 7 |
| 3   | 2 6                         |
| 6   | 3                           |

ans = 4 2

O/p: 2

TC  $\Rightarrow O(N)$   
SC  $\Rightarrow O(N)$

map<int, int>

for (i = 0  $\rightarrow$  N-1) {

if (arr[i] is present in map) {

ans = min(ans, i - HM(arr[i]))

update map (arr[i], i)

}

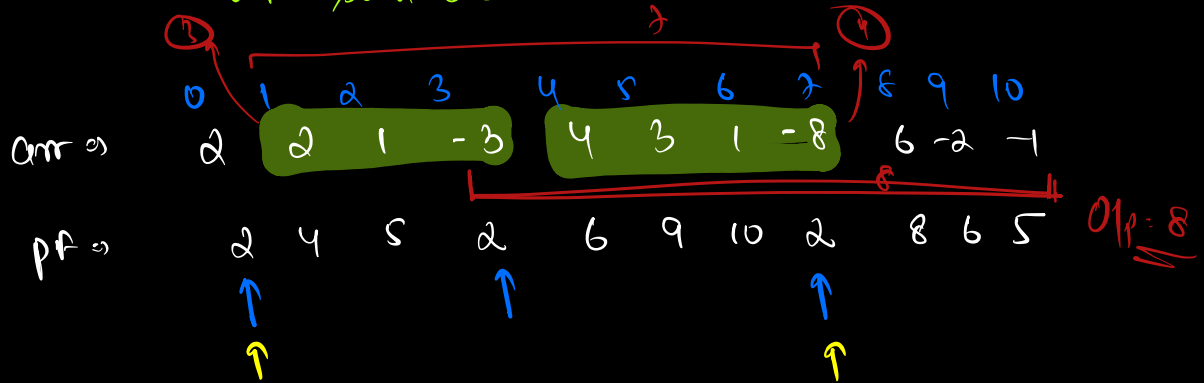
else {

insert map (arr[i], i)

}

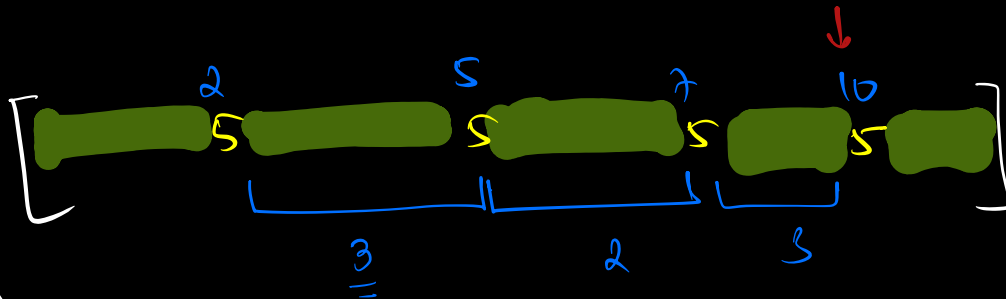
}

Q2. Given an integer array, find the longest subarray with sum = 0.



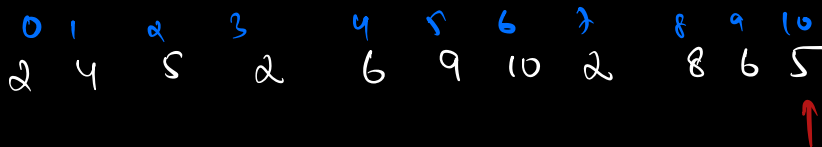
$$(j-i) \Rightarrow pf[j] - pf[i-1] = 0$$

$$\Rightarrow pf[j] = pf[i-1]$$



TC =  $O(N)$   
SC =  $O(N)$

Opt = 2



ans = 3

| element | idx |    |   |
|---------|-----|----|---|
| 2       | 0   | 6  | 4 |
| 4       | 1   | 9  | 5 |
| 5       | 2   | 10 | 6 |
|         |     | 8  | 8 |



Q3. Given an arr[N], find the length of largest sequence which can be rearranged to get consecutive elements -

ex  $\Rightarrow$  100 4 3 6 10 20 11 5 101

100  $\rightarrow$  101  $\rightarrow$  ~~102~~  $\rightarrow$  (2) 0/p = 4

4  $\rightarrow$  5  $\rightarrow$  6  $\rightarrow$  ~~7~~  $\rightarrow$  (3)

[3  $\rightarrow$  4  $\rightarrow$  5  $\rightarrow$  6]  $\rightarrow$  ~~7~~  $\rightarrow$  (4)

6  $\rightarrow$  ~~7~~  $\rightarrow$  (1)

10  $\rightarrow$  11  $\rightarrow$  ~~12~~  $\rightarrow$  (2)

20  $\rightarrow$  ~~21~~  $\rightarrow$  (1)

11  $\rightarrow$  ~~12~~  $\rightarrow$  (1)

5  $\rightarrow$  6  $\rightarrow$  ~~7~~  $\rightarrow$  (2)

101  $\rightarrow$  ~~102~~  $\rightarrow$  (1)

So  $\Rightarrow$  3 4 5 6 10 11 20 100 101

$\xrightarrow{\quad} \xrightarrow{\quad} \xrightarrow{\quad}$   
~~7~~ ~~12~~ ~~17~~  
 $\xrightarrow{\quad}$   
 4

and check

$TC \Rightarrow O(N \log N)$   
 $SC \Rightarrow O(N)$

\* considers every element as a starting pt.

arr = [-1 8 2 3 7 1 4 9]

$O(N^3)$

-1 → ~~X~~ → ①  
~~8 → 9 → ~~X~~ → ①~~  
~~2 → 3 → 4 → ~~X~~ → ③~~  
~~3 → 4 → ~~X~~ → ②~~  
7 → 8 → 9 → ~~X~~ → ③  
1 → 2 → 3 → 4 → ~~X~~ → ④  
~~4 → ~~X~~ → ①~~  
~~9 → ~~X~~ → ①~~

O/p = 4

Optimize search using Hashset  
TC =  $O(N^2)$

if, for element [arr[i]] if arr[i]-1 is present in array, arr[i] can't be the starting point.

-1 8 2 3 7 1 4 9

ans = ~~4~~ 3

-1, 8, 2  
3, 7, 1  
4, 9

TC =  $O(N)$   
SC =  $O(N)$

|    |                |
|----|----------------|
| -1 | 0 <del>x</del> |
| 8  | 7 ✓            |
| 2  | 1 ✓            |
| 3  | 2 ✓            |
| 7  | 6 <del>x</del> |
| 1  | 0 <del>x</del> |
| 4  | 3 ✓            |
| 9  | 8 ✓            |

-1 → ~~0~~  
8 can't be starting  
2 " " "  
3 " " "  
7 → 8 → 9 → ~~1~~  
1 → 2 → 3 → 4 → ~~5~~  
4 can't be start  
9 " " "

ans = 6 6 6 6 6 6 7 8 9

6 7 8 9

6 7 8 9

6 7 8 9

6 7 8 9 —————

1) Create the hashtable and insert all elements into it—

1) Iterate over hashtable—

$x$

if ( $x-1$  doesn't exist in hashtable) {

chain = 1

$y = x-1$

while ( $y$  is present in set) {

chain++

$y++$

}

ans = max(ans, chain)

}

)

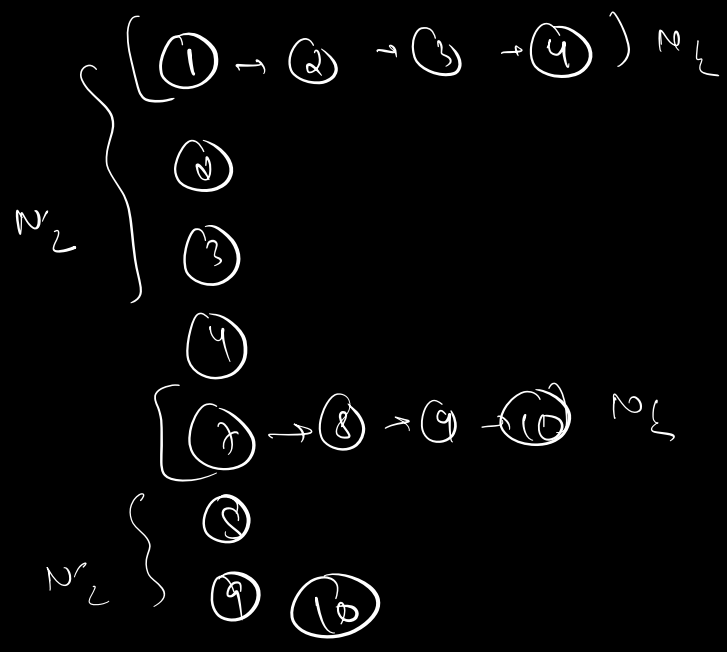
$TC = O(N)$   
 $SC = O(N)$

1 2 3 4 5 6 7

① → ② → ③ → ④ → ⑤ → ⑥ → ⑦

② ③ ④ ⑤ ⑥ ⑦

1 2 3 4 7 8 9 10



$$\rightarrow 2N \approx \frac{2}{1}$$